

Chap. 17

Designing Boundary Classes

Object Oriented Systems Analysis and Design Using UML, (4th Edition),
McGraw Hill

Topics Covered

- Presentation Layer
- Prototyping User Interface
- Designing Classes
- Designing Interaction with Sequence Diagrams
- Modelling the Interface Using State Machines

Presentation Layer

- 3-Tier Architecture
 - Boundary class, Control class , Entity class
- Boundary class (Presentation layer)
 - Handles interface with users and other systems
 - Provides a mechanism for data entry by the user
 - Formats and presents data at the interface

Developing Boundary Classes

- ① Prototype the user interface
- ② Design the classes (attributes)
- ③ Model the interaction involved in the interface (operations)
- ④ Model the control of the interface using state machines (if necessary)

Prototyping the User Interface

- A prototype is a model that looks, and partly behaves, like the finished product but lacks certain features

Check Campaign Budget

Use Case Prototype

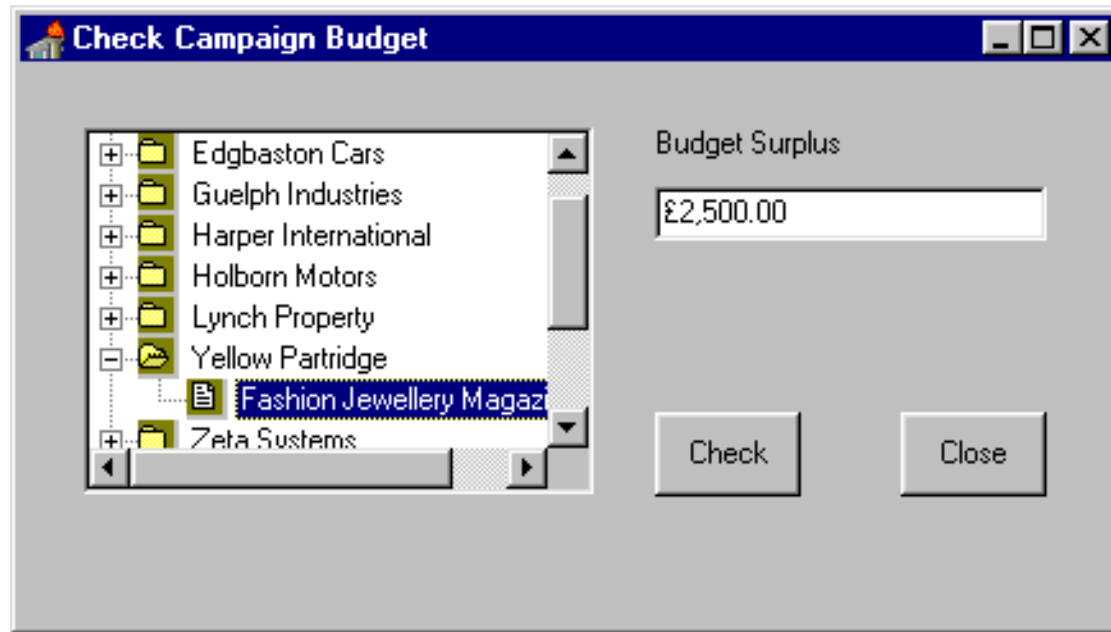


The image shows a software dialog box titled "Check Campaign Budget". It contains three input fields arranged vertically. The first field is labeled "Client" and is a dropdown menu with "Yellow Partridge Jewellery" selected. The second field is labeled "Campaign" and is a dropdown menu with "Fashion Jewellery Magazine" selected. The third field is labeled "Budget Surplus" and is a text input field containing the value "£2,500.00". At the bottom of the dialog, there are two buttons: "Check" on the left and "Close" on the right.

- In this prototype, Clients and Campaigns are selected in drop-down lists
- There are other ways...

Check Campaign Budget

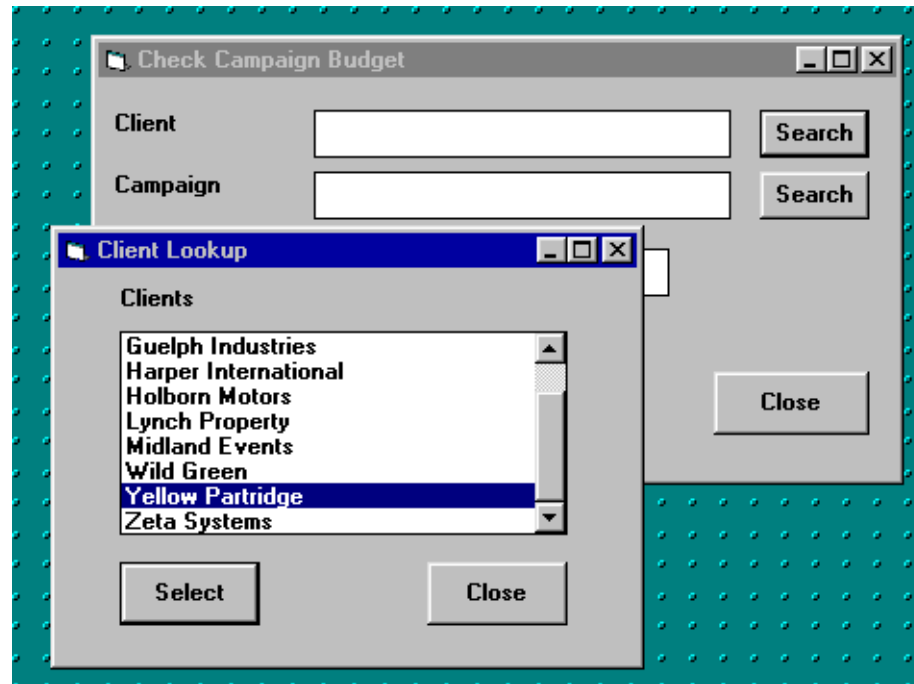
Use Case Prototype



- Using a treeview control...

Check Campaign Budget

Use Case Prototype

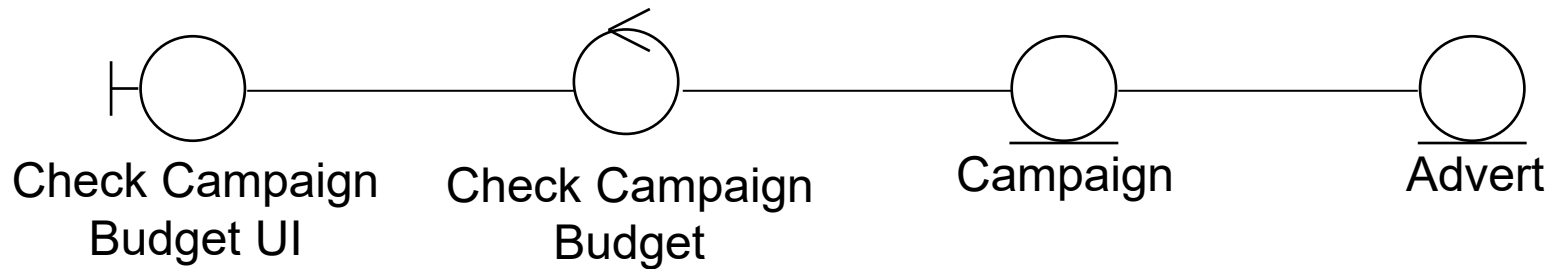


- Using a separate look-up window

Designing Classes

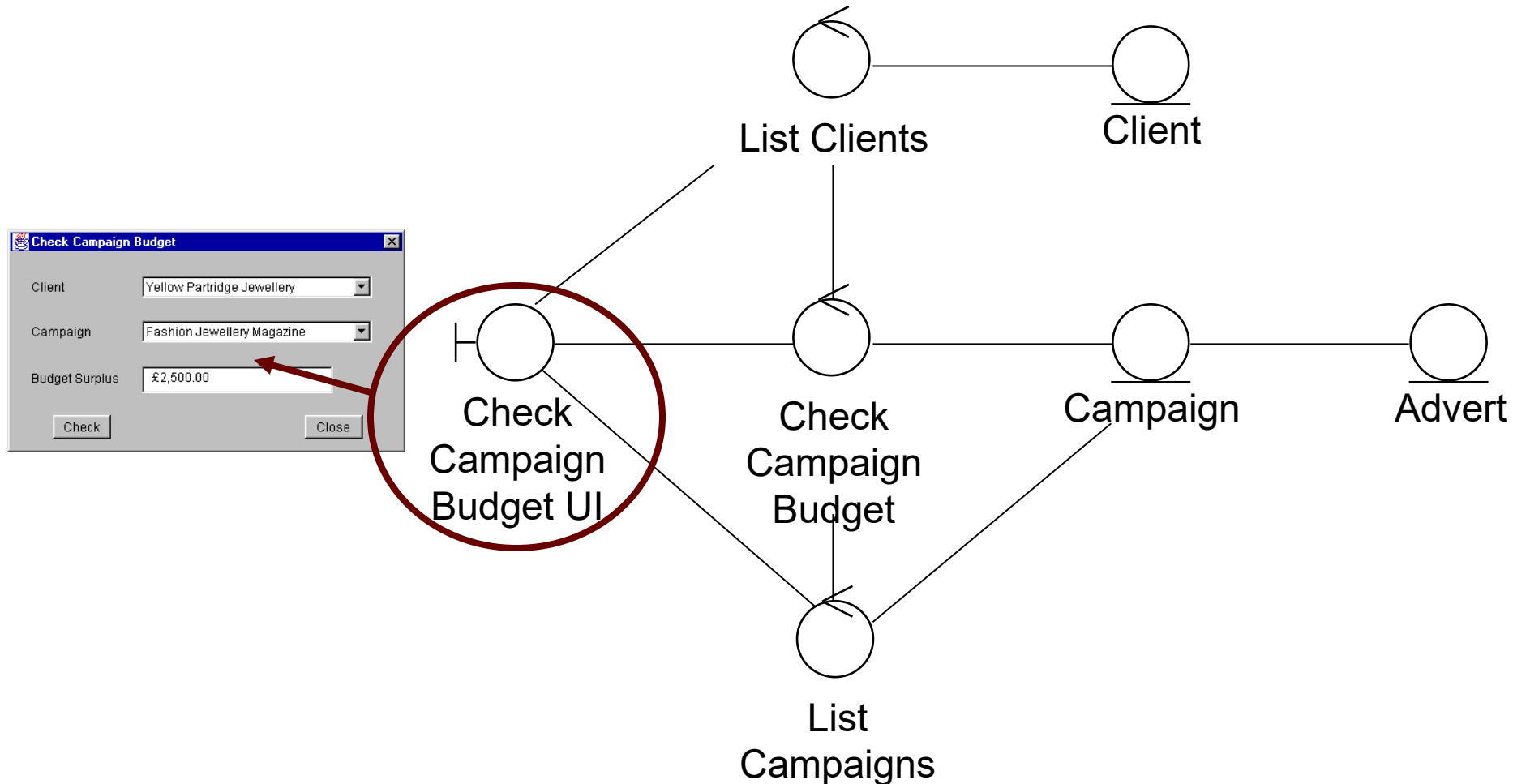
- Start with the collaborations from the analysis model
- Elaborate the collaborations to include necessary boundary, entity and control classes

Collaboration for Check campaign budget

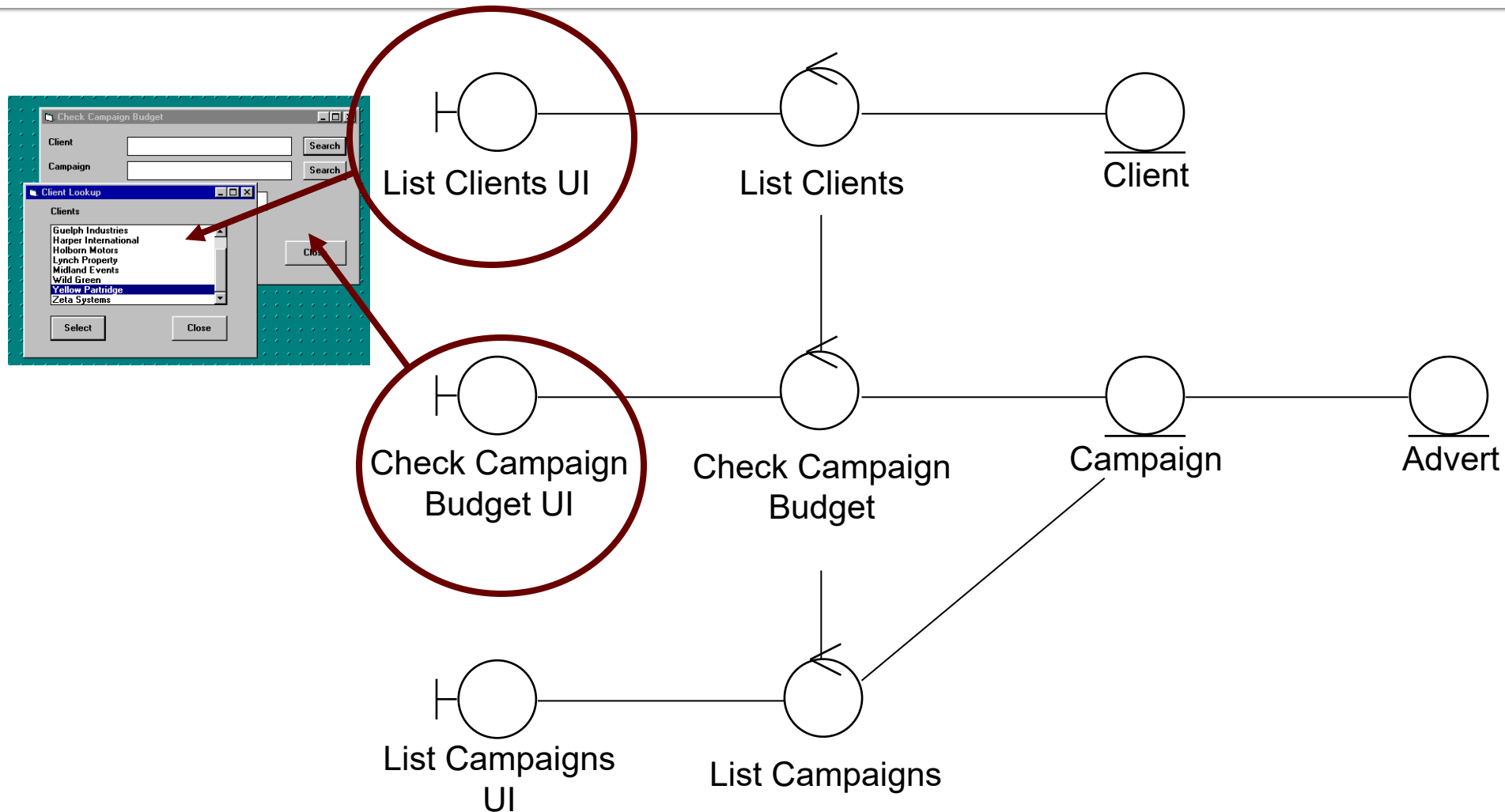


- In order to find the right Campaign, we also need to use the Client class
- We also add control classes for the responsibilities of listing clients and campaigns

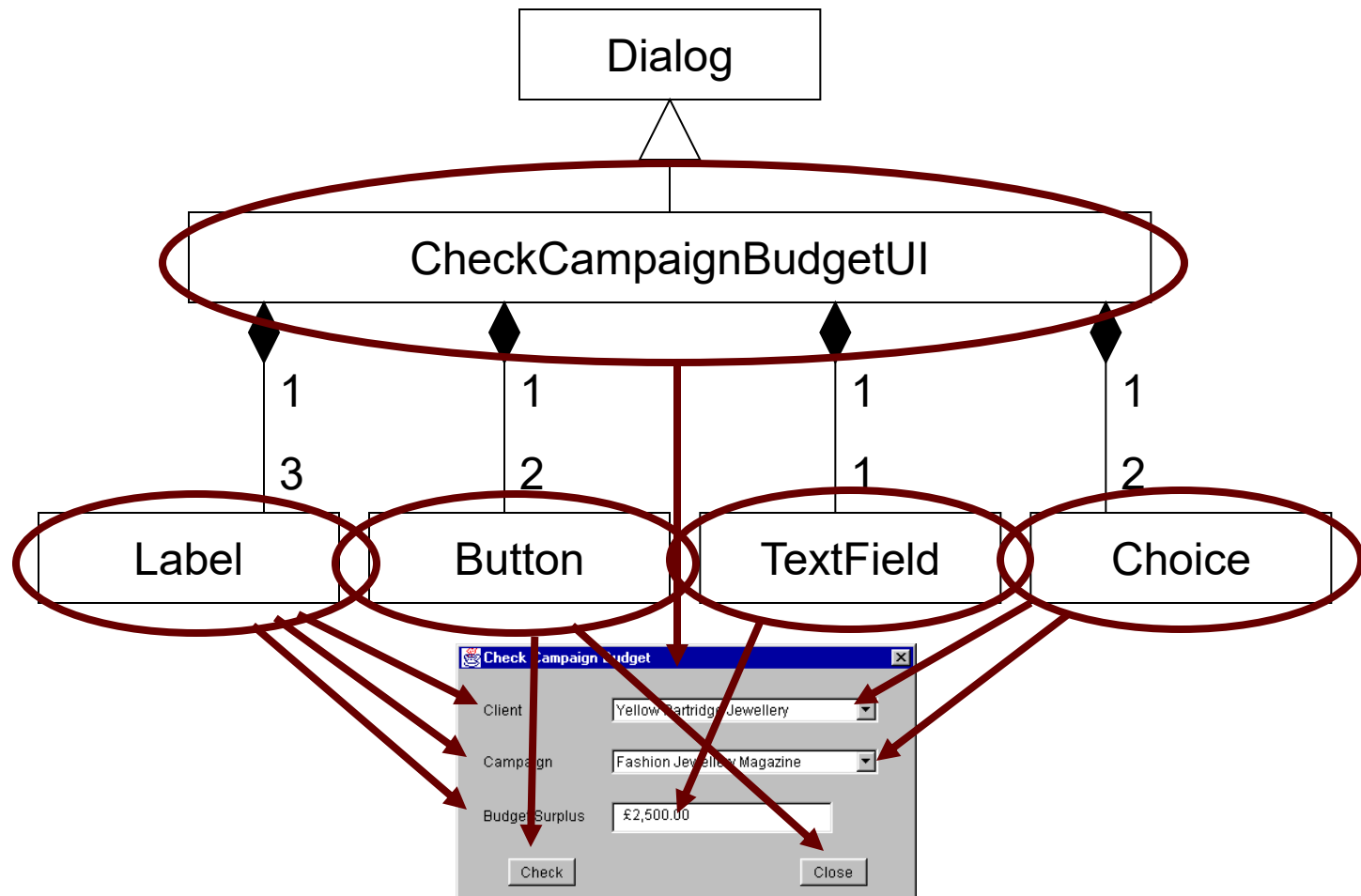
Collaboration for Check campaign budget



Alternative Collaboration for Check campaign budget



Class Diagram for CheckCampaignBudgetUI



Single Class for CheckCampaignBudgetUI

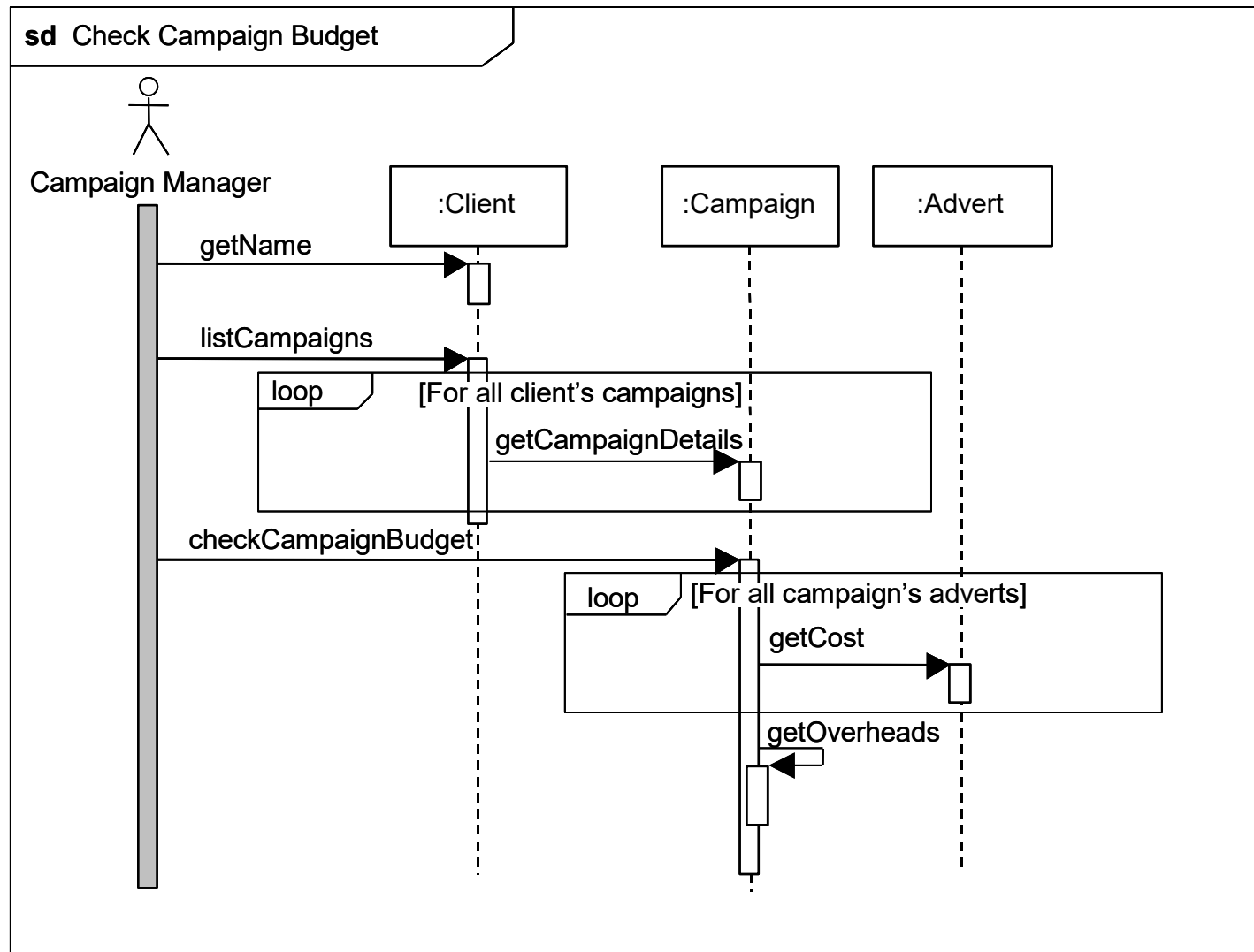
- Draw in your own lines to show which attribute is which element of the interface



CheckCampaignBudgetUI

- clientLabel : Label
- campaignLabel : Label
- budgetLabel : Label
- checkButton : Button
- closeButton : Button
- budgetTextField : TextField
- clientChoice : Choice
- campaignChoice : Choice

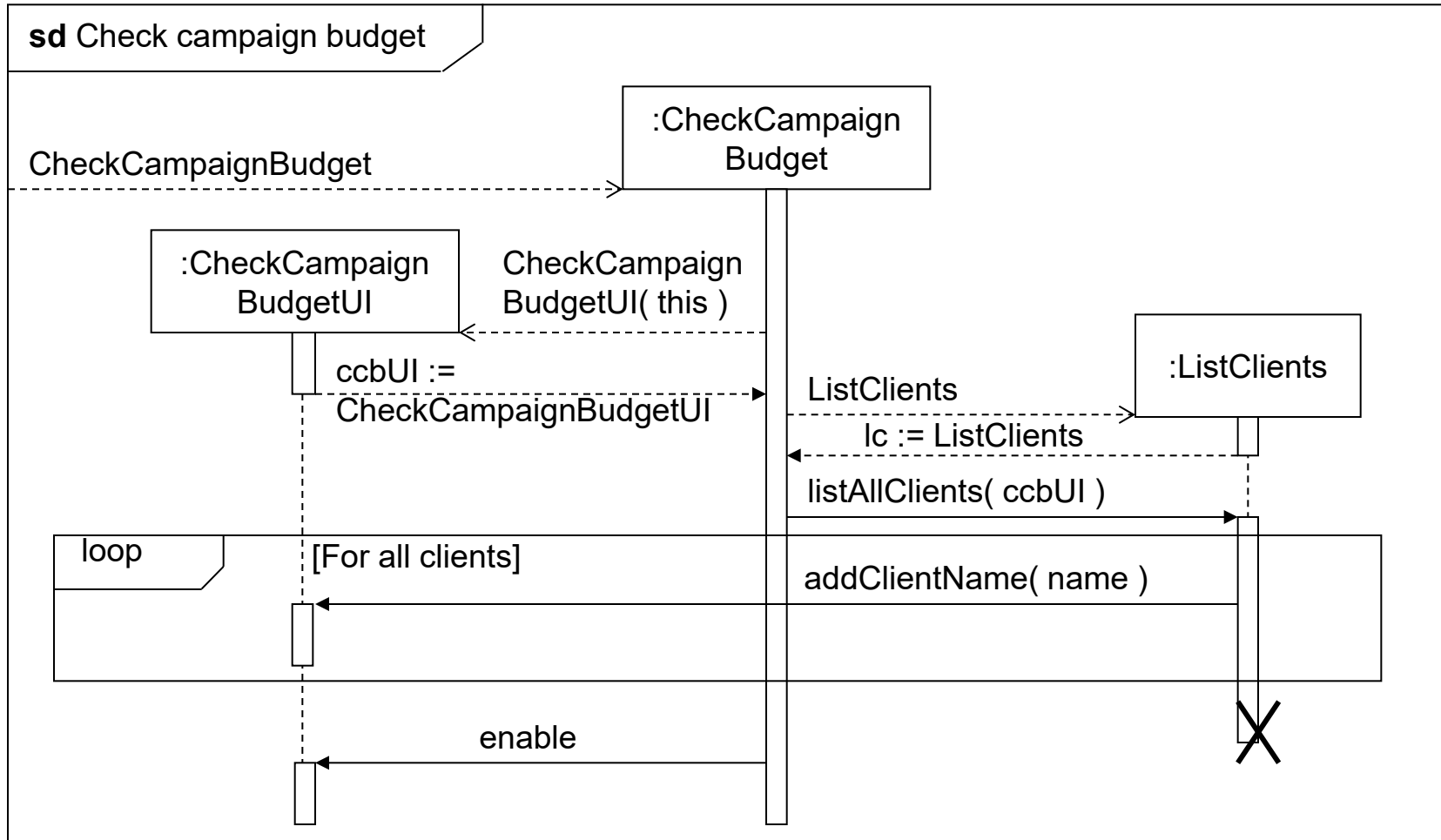
Designing Interaction with Sequence Diagrams



Sequence Diagrams

- The sequence diagram on the previous slide just shows the entity classes
- We also need to model the interaction more detail
 - By showing the control and boundary classes

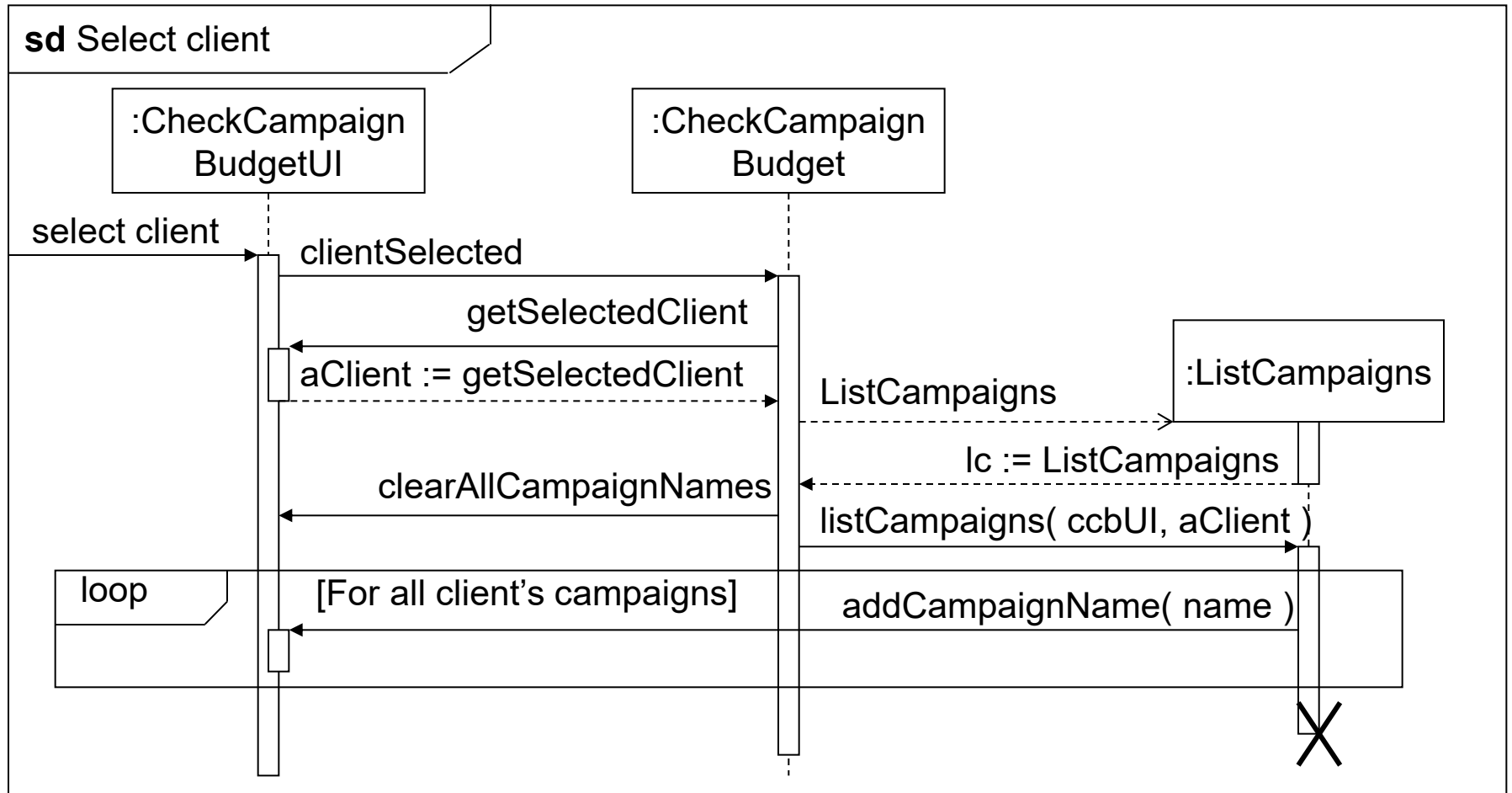
First Part of Sequence Diagram



First Part of Sequence Diagram

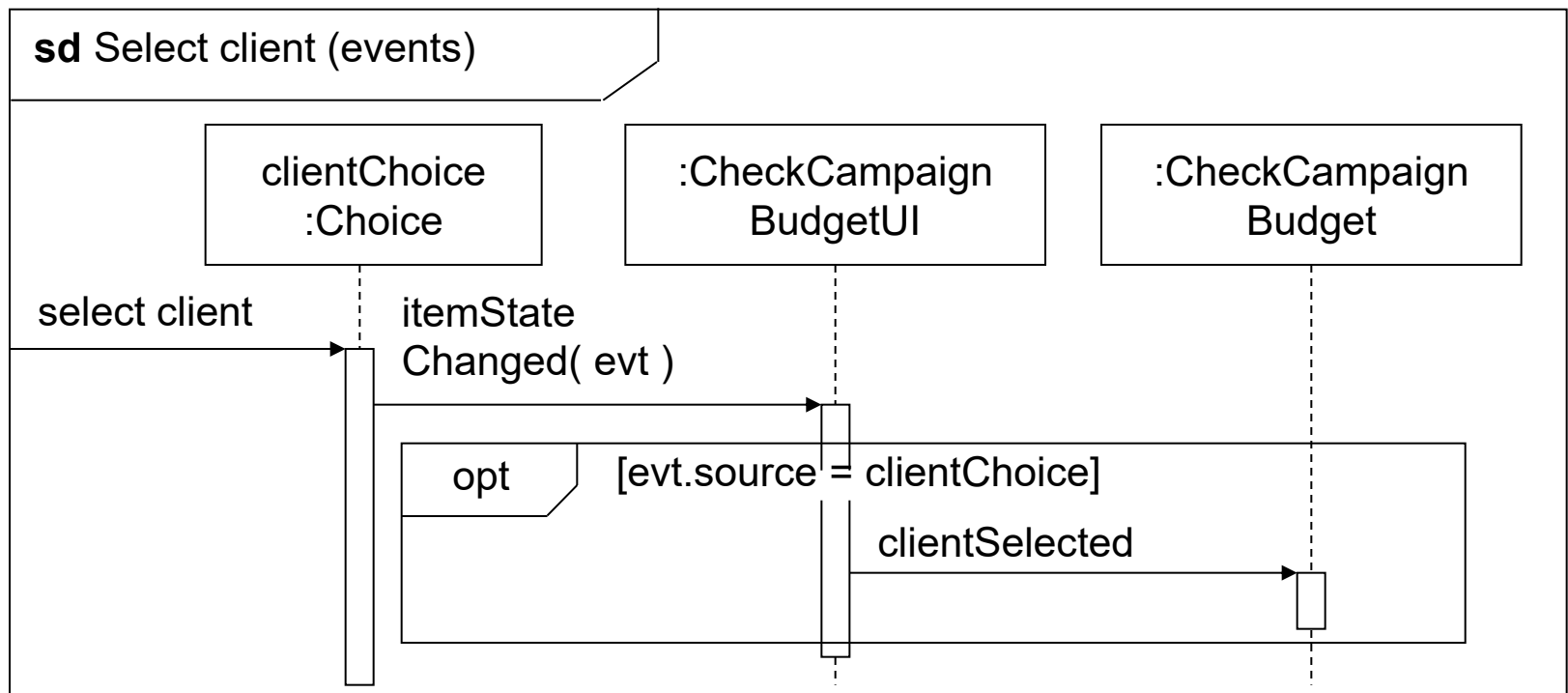
- The control class
 - creates the instance of the boundary class
 - creates the instance of the **ListClients** control class
 - passes to :ListClients a reference to the boundary class
 - :ListClients then sets the name of each client in turn into the boundary class by calling **addClientname (name)** repeatedly in the loop

Second Part of Sequence Diagram

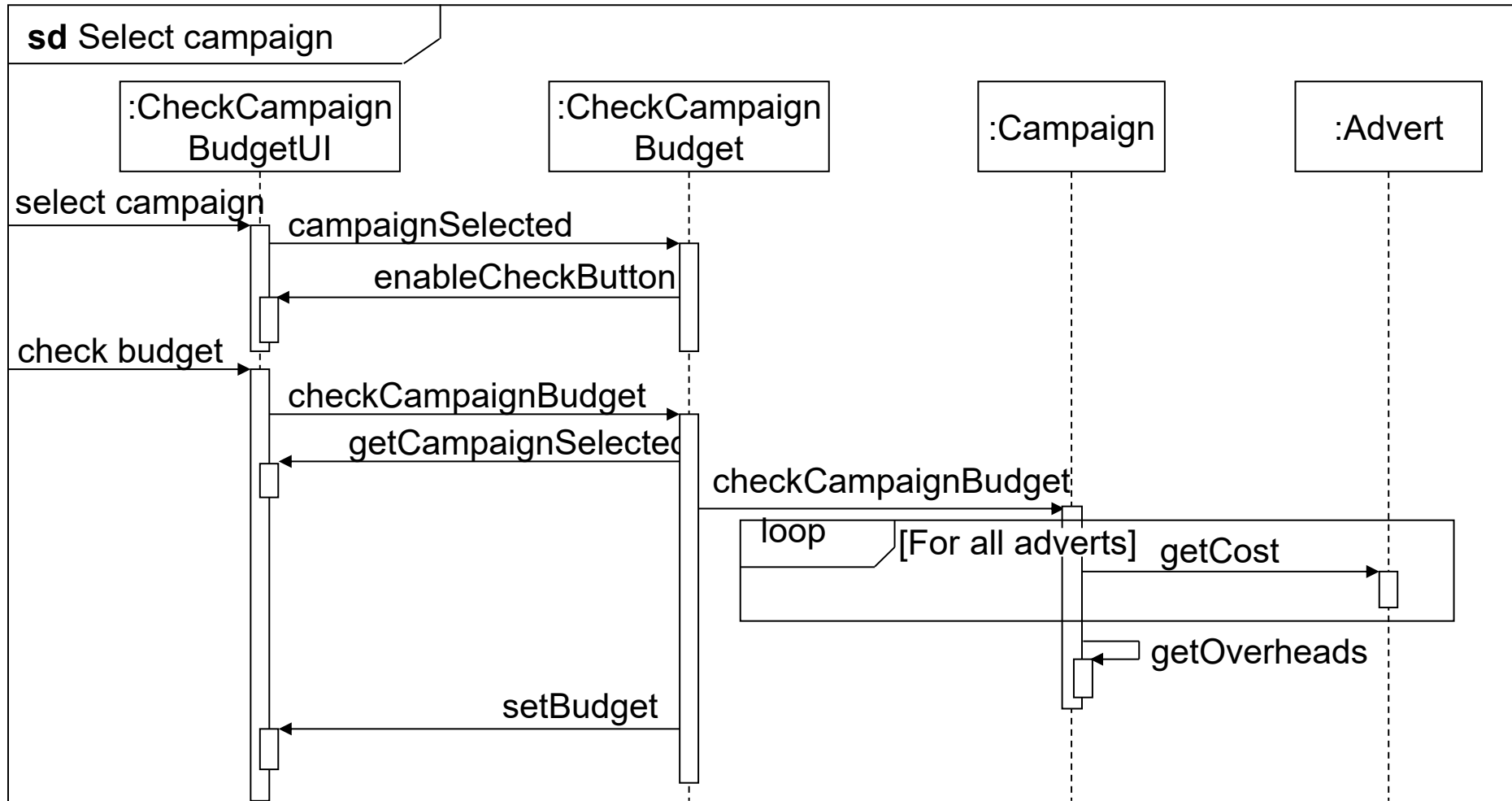


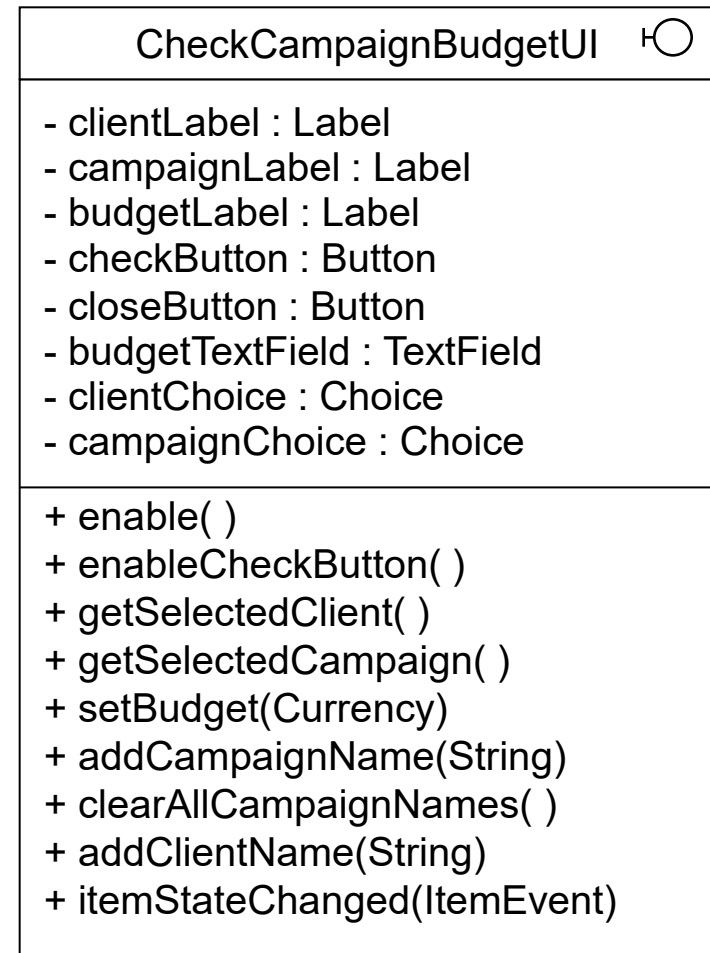
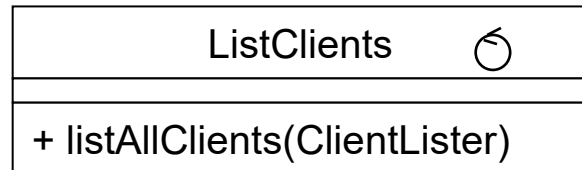
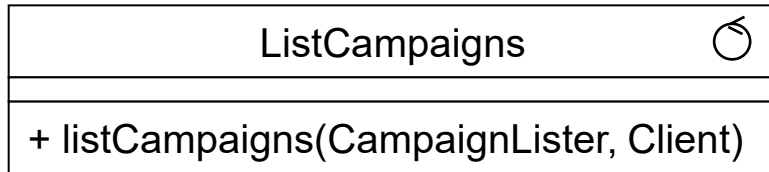
Event-driven User Interface

Event in :clientChoice is passed through to the main boundary class



Final Part of Sequence Diagram





Modelling the User Interface in State Machines

- Five tasks of Horrocks' approach:
 - describe the high-level requirements and main user tasks
 - describe the user interface behaviour
 - define user interface rules
 - draw the state machine (and successively refine it)
 - prepare an event action table

High-level Requirements

- The requirement here is that the users must be able to check whether the budget for an advertising campaign has been exceeded or not.
- This is calculated by summing the cost of all the adverts in a campaign, adding a percentage for overheads and subtracting the result from the planned budget.
- A negative value indicates that the budget has been overspent. This information is used by a campaign manager.

User Interface Behaviour

- The **client dropdown** displays a list of clients. When a client is selected, their campaigns will be displayed in the campaign dropdown.
- The **campaign dropdown** displays a list of campaigns belonging to the client selected in the client dropdown. When a campaign is selected the check button is enabled.
- The **budget textfield** displays the result of the calculation to check the budget.
- The **check button** causes the calculation of the budget balance to take place.
- The **close button** closes the window and exits the use case.

Define User Interface Rules

- User interface objects with constant behaviour
 - The client dropdown has constant behaviour. Whenever a client is selected, a list of campaigns is loaded into the campaign dropdown
 - The budget textfield is initially empty. It is cleared whenever a new client is selected or a new campaign is selected. It is not editable
 - The close button may be pressed at any time to close the window

Define User Interface Rules

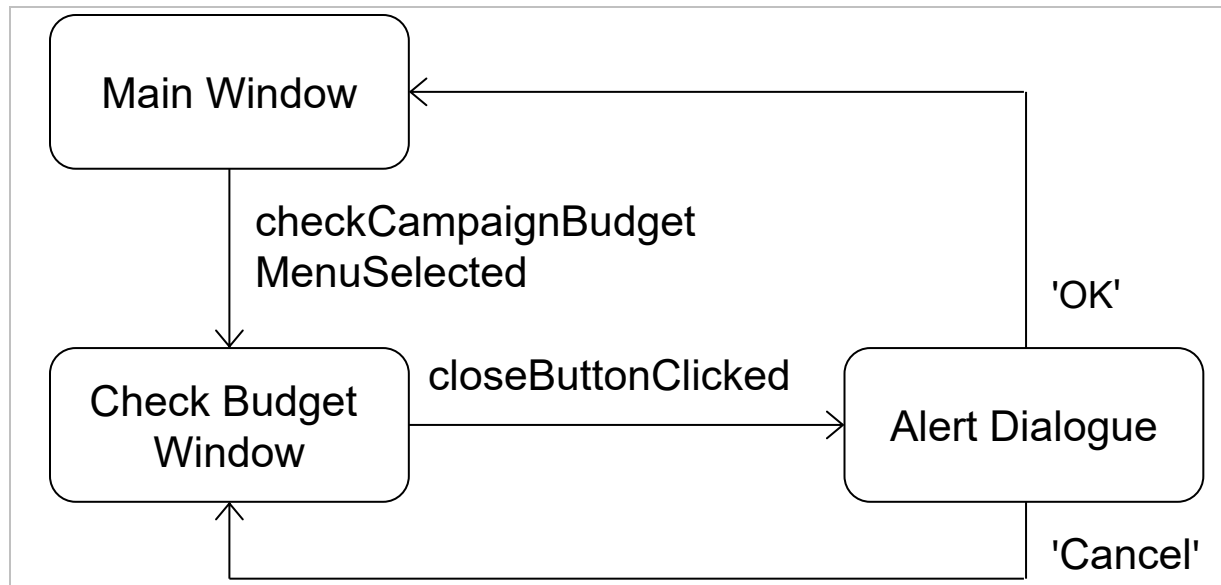
- User interface objects with varying behaviour
 - The campaign dropdown is initially disabled. No campaign can be selected until a client has been selected. Once it has been loaded with a list of campaigns it is enabled
 - The check button is initially disabled. It is enabled when a campaign is selected. It is disabled whenever a new client is selected

Define User Interface Rules

- Entry and exit events
 - The window is entered from the main window when the **Check Campaign Budget** menu item is selected
 - When the close button is clicked, an alert dialogue is displayed. This asks 'Close window? Are you sure?' and displays two buttons labelled 'OK' and 'Cancel'. If 'OK' is clicked the window is exited; if 'Cancel' is clicked then it carries on in the state it was in before the close button was clicked

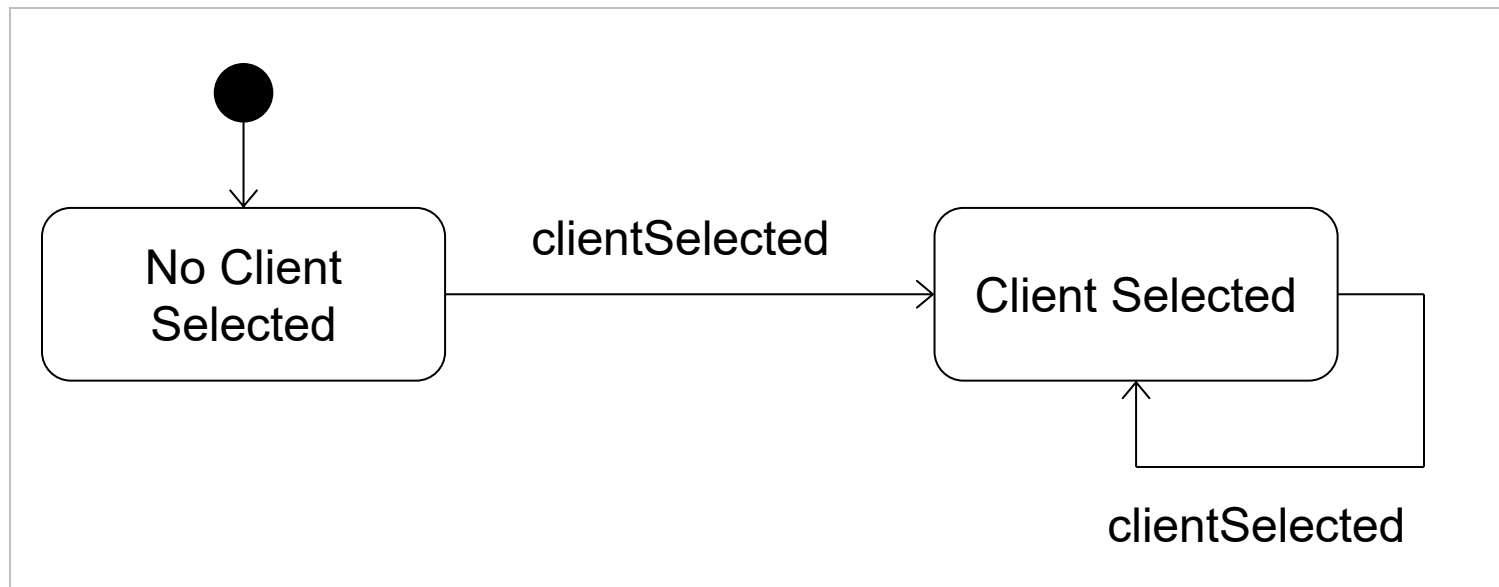
Draw the State Machine

- We start with the top-level state machine for movement between the windows and dialogues



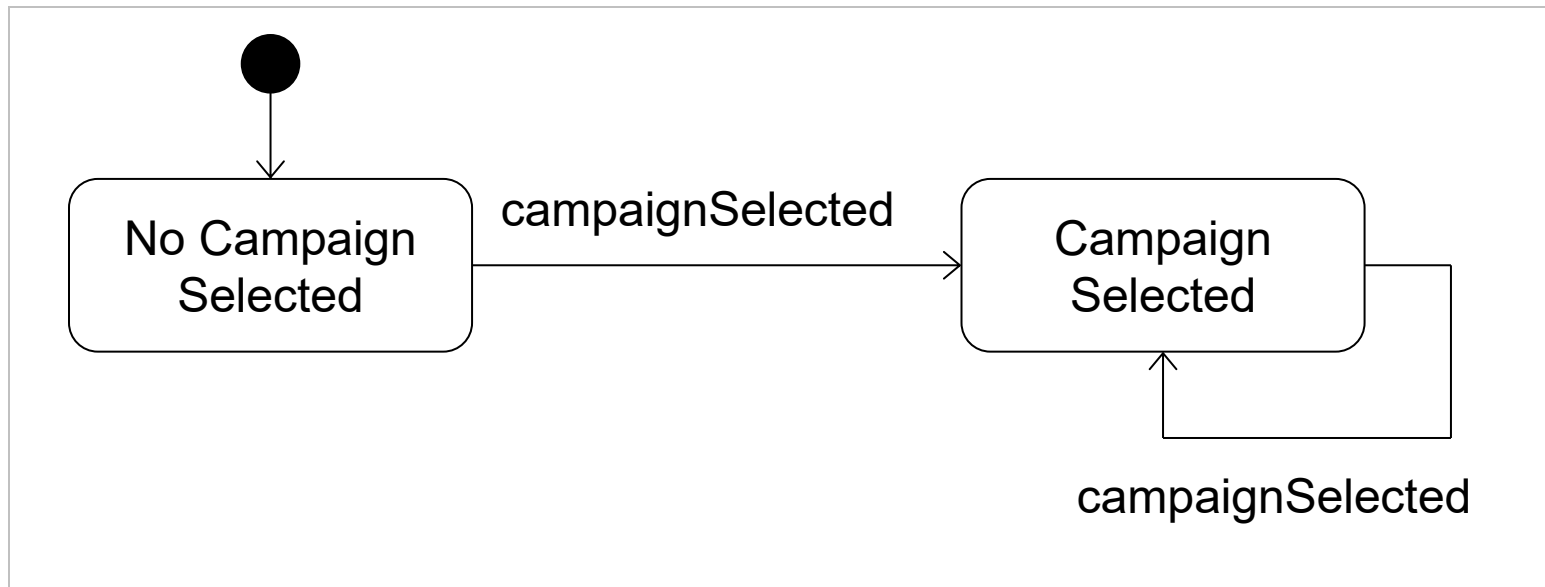
Draw the State Machine

- Client selection states are nested within the **Check Budget Window** state



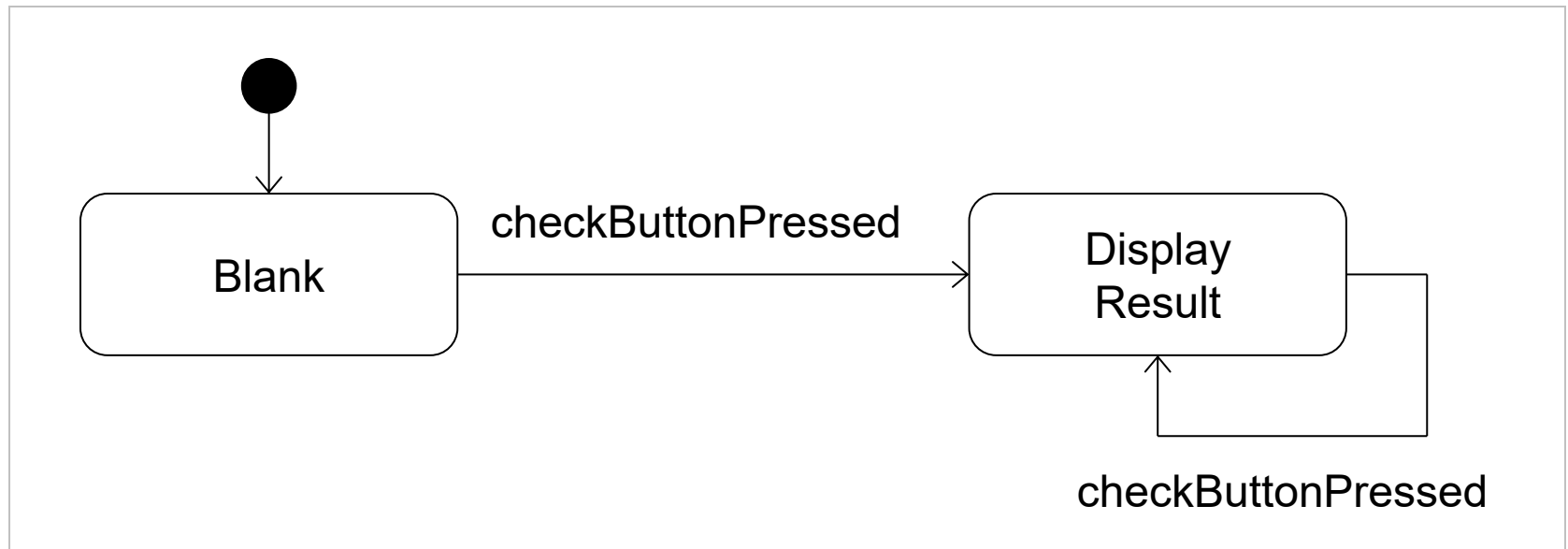
Draw the State Machine

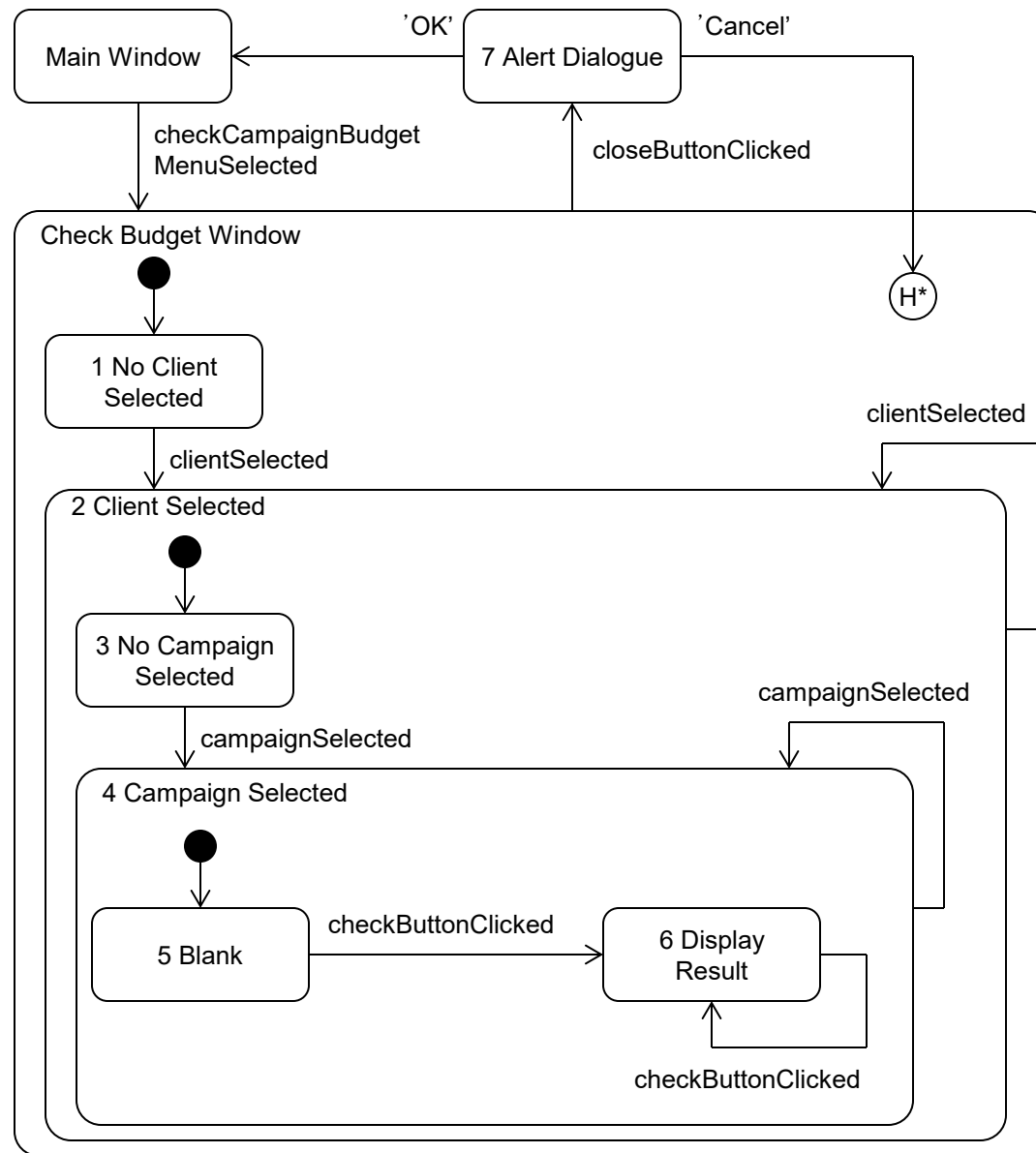
- Campaign selection states are nested within the **Client Selected** state



Draw the State Machine

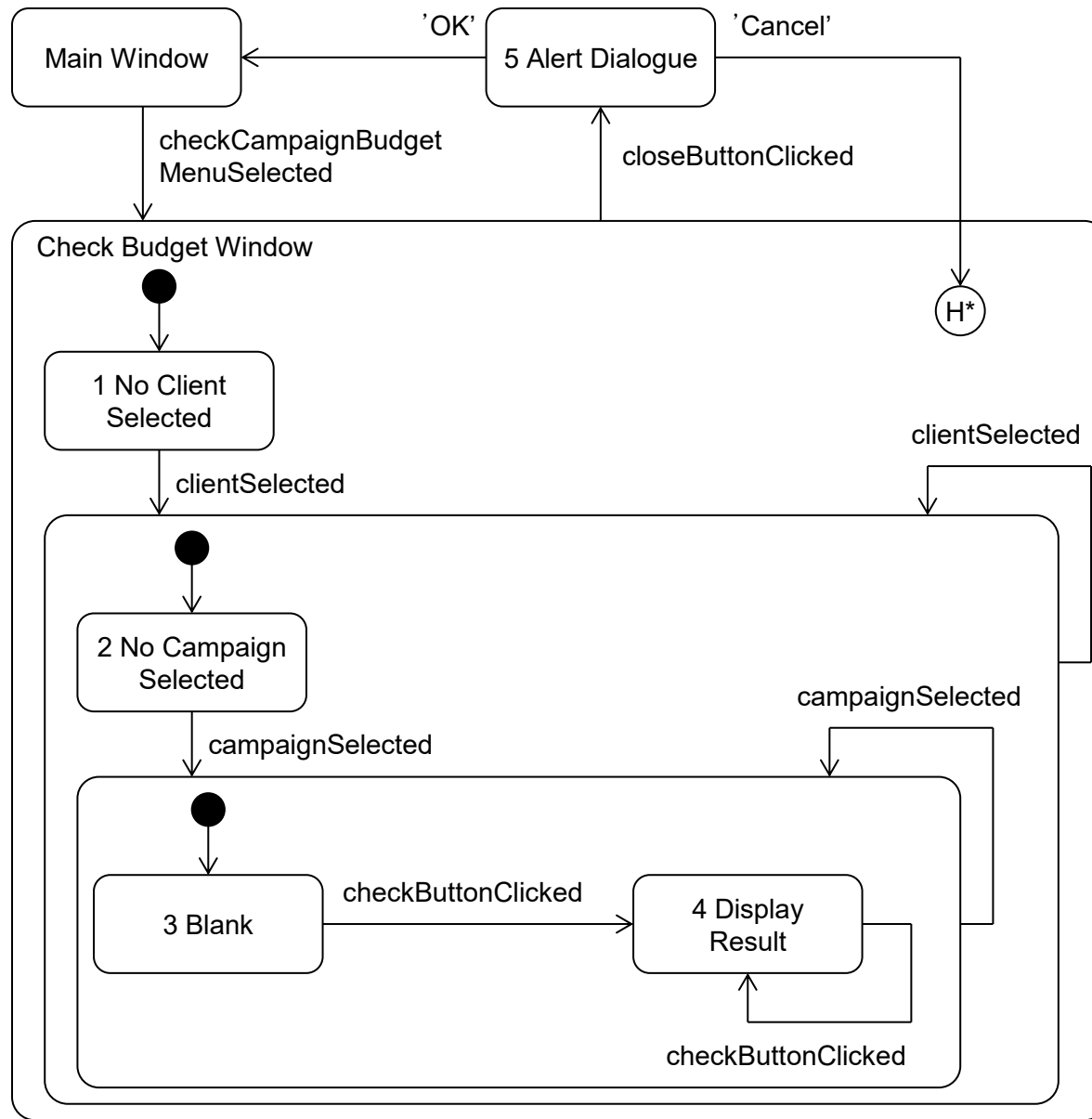
- Display of result states are nested within **Campaign Selected** state





Draw the State Machine

- State machine can be simplified (as on next slide)
- Rather than try to add all messages associated with transitions into the diagram, an Event–Action table can be used



Event–Action Table

Current State	Event	Action	Next State
–	Check Campaign Budget menu item selected.	Display <u>CheckCampaignBudgetUI</u> . Load client dropdown. Disable campaign dropdown. Disable check button. Enable window.	1
1	Client selected.	Clear campaign dropdown. Load campaign dropdown. Enable campaign dropdown.	2
2, 3, 4	Client selected.	Clear campaign dropdown. Load campaign dropdown. Clear budget <u>textfield</u> . Disable check button.	2
2	Campaign selected.	Clear budget <u>textfield</u> . Enable check button.	3
3	Check button clicked.	Calculate budget. Display result.	4
3, 4	Campaign selected.	Clear budget <u>textfield</u> .	3
4	Check button clicked.	Calculate budget. Display result.	4
1, 2, 3, 4	Close button clicked.	Display alert dialogue.	5
5	OK button clicked.	Close alert dialogue. Close window.	–
5	Cancel button clicked.	Close alert dialogue.	H*

Revising the Sequence Diagrams and Class Diagrams

- Producing the state machine and the event–action table has identified some additional messages that will be sent to the user interface to control it
- These will need to be added to the sequence diagrams and to the class diagram as operations of the UI class or of the lister interfaces

Revised First Sequence Diagram

